

實驗作品名稱:貪食蛇

實驗目的&內容:

1. 實驗目的:要在8051微控制器實習板上,透過板上的連接埠、按鍵8*8點陣顯示器,做對應連接插線後,在電腦上的SIMLAB程式做貪食蛇組合語言撰寫,組合語言內容包含"遊戲一開始之蛇、隨機食物、固定障礙物,還有相關遊戲規則,以及貪食蛇移動控制",將程式碼輸出至板子後就會在實習板上顯示貪食蛇之遊戲互動介面,操作者可藉此做貪食蛇相關的操作,本實驗作品即為本學期為算機實驗課程之其中驗收作品。
2. 實驗內容:
 - (1) 程式碼(組合語言)要做到的內容方法及規則:

內容:

 - 蛇的初始位置生成、食物隨機位置生成、固定之牆面生成。
 - 點陣亮燈部分,蛇的初始身長是一顆,一旦吃到一顆食物就會加長身長一顆;食物自始自終皆為一顆,再隨機生成食物上,食物只會有一顆再8*8點陣顯示器上,吃完一顆才會出現下一顆;障礙物為三組兩顆作為固定牆面障礙物。
 - 按鍵要做到可以控制蛇的移動之上下左右,以及蛇的移動速度可以做到變快或是變慢。
 - 詳細方法:
 - 主程式架構(MAIN + GAME_LOOP):
 - a. 程式一開始從 `ORG 0000H` 跳到主程式 MAIN,使用 `LJMP MAIN`。
 - b. 主流程由 `GAME\_LOOP` 負責不斷循環執行,主要功能包含:顯示畫面(呼叫 `DISPLAY\_ALL`),偵測按鍵(呼叫 `CHECK\_KEY`),控制蛇的移動(呼叫 `MOVE\_SNAKE`),判斷是否遊戲結束(`gameOver` 狀態)。
 - c. 一開始會先執行 `INIT\_GAME` 初始化所有遊戲資料,包含:蛇的起始長度與位置(存在 RAM 的 30H 開始),初始方向變數(例如 X, Y 向量),食物初始座標,Timer0 設定成亂數用(用來產生食物位置用的種子值)。
 - 畫面顯示控制(DISPLAY_ALL 與 DISPLAY_DOT):
 - a. DISPLAY_ALL 副程式負責整體畫面的掃描顯示,每一輪會依序顯示:蛇的身體、食物位置、障礙物。
 - b. 顯示 LED 的方式是利用 `DISPLAY\_DOT`,透過位移暫存器與行列控制:行訊號透過 P1.0~P1.7,列訊號透過 P0.0~P0.7。利用 `RL` (Rotate Left)操作來選擇對應的行,再用 `CPL` (Complement)反相方式控制亮/滅。
 - c. 顯示時會有短暫延遲,並不會一次亮整片,而是高速掃描達成穩定視覺顯示。
 - d. 障礙物的圖案資料則是寫在 CODE 區的 `BLOCK\_DATA`,顯示時會將這些資料一起畫出,讓畫面中同時顯示蛇、食物、障礙物。
 - 按鍵偵測 (CHECK_KEY)
 - a. CHECK_KEY 會不斷讀取按鍵輸入,更新蛇的「下一步方向」變數(`nextDirX`、`nextDirY`)。
 - b. 為了避免誤觸或反方向撞牆的問題,按鍵更新並不會馬上生效,而是要等到下一次 Timer 週期到時才會生效。
 - c. 若偵測到方向改變,會暫存到變數中,等待下一個移動更新週期應用。
 - d. 程式會利用 16-bit 計時器(48H、49H)調整速度,越小的延遲值代表蛇移動越快。
 - 蛇的移動邏輯(MOVE_SNAKE):
 - a. MOVE_SNAKE 是主要的遊戲邏輯:計算新蛇頭座標(`newHeadX`, `newHeadY`),判斷是否越界(超出 8x8 範圍),檢查是否咬到自己(撞身體),若發生這些情況,會將 `gameOver` 設為 1。
 - b. 如果蛇頭座標與食物位置重疊,就代表吃到食物:蛇長度會加 1,呼叫 `NEW\_FOOD` 重新生成食物座標。
 - c. 若沒有吃到食物,就讓蛇的身體往前推移:將每一節的座標往前一格複製,更新蛇頭的新位置,尾巴就會自動清除掉。
 - 食物生成副程式(NEW_FOOD):
 - a. 這個副程式會使用 Timer0 的 TH0 與 TL0 當亂數種子,把兩者相加後再用 `ANL #07H` 取低三位元,確保座標在 0~7 範圍內。
 - b. 程式會反覆檢查生成的座標是否與蛇身或障礙物重疊,若重疊則重新生成。
 - c. 直到找到一個合法位置為止,才會將座標存入食物的 X/Y 變數中。
 - d. 生成完成後,食物會在下一次 `DISPLAY\_ALL` 更新時被畫出。
 - 遊戲結束與閃爍顯示(GAME_OVER)
 - a. 當 `gameOver` 被設定為 1,程式會跳進 `GAME\_OVER` 狀態。
 - b. 在這個階段,畫面會進入閃爍模式:全亮(P0/P1 全亮) -> 全暗 -> 再全亮 -> 如此重複幾次。
 - c. 閃爍的目的就是顯示"遊戲結束"。
 - d. 此時所有變數都會凍結,直到使用者按下 RESET 鍵,程式才會重新初始化,回到 `MAIN` 開始新的一局遊戲。
 - 補充:
 - a. 計時器 (`Timer0`) 不僅控制移動節奏,還同時當亂數來源使用。
 - b. 蛇身座標採「雙 byte」方式儲存(例如 30H 開始,依序存 X、Y)。

- c. 顯示方式為逐行掃描，並非一次輸出整個 8×8。
- d. 蛇、食物、牆壁的資料都在 RAM 中維護，唯有障礙物在 ROM 的 CODE 區中。
- e. gameOver 是用旗標 (bit) 控制流程，不直接跳出主程式。

主要規則:

- 蛇的移動速度大約是每一秒往前移動一格(一顆燈)的距離。
- 蛇一但碰到障礙物(三組兩顆燈)、碰到8*8點陣之邊緣(超過點陣之範圍)，或是自身身體，即刻GAME OVER。

(2) 連接埠和功能對應:

連接埠(功能)	連接埠(port)
8*8點陣_ROW:R1~R8	P11~P17
8*8點陣_LOW:C1~C8	P18~P26
上:S5	P39
下:S6	P40
左:S7	P41
右:S8	P42
加速:S3	P37
減速:S4	P38
重新開始:S2	P36

(3) 內容對應RAM記憶體之位置配置:

儲存蛇的每節x座標:snakeX[0]~snakeX[31]	50H~6FH
儲存蛇的每節y座標:snakeY[0]~snakeY[31]	70H~8FH
食物座標:foodX、foodY	40~41H
蛇的長度	42H
目前&預備方向:dirX、dirY & nextDirX、nextDirY	43H~46H
遊戲結束之旗標:gameOver	47H
速度之參數:16-bit	48H~49H
計數器:t	4AH~4BH

貪食蛇組合語言(code):

; 一開機直接跳到 MAIN (主程式入口)	
LJMP MAIN	; 一開機直接跳到 MAIN (主程式入口)
; 主程式	
MAIN:	; 主程式標籤
MOV SP, #60H	; 設定堆疊指標，避免跟我們的資料區打架
ACALL INIT_GAME	; 先把遊戲資料初始化 (蛇、食物、速度...)
GAME_LOOP:	; 主迴圈開始
; 檢查 RESET 鍵 (P3.1)	; 先看有沒有按下重設
JB P3.1, NO_RESET	; P3.1=1 表示沒按 (接上拉)，跳去 NO_RESET
ACALL INIT_GAME	; 重新初始化遊戲
SJMP WAIT_RESET_RELEASE	; 有按下就整個重來
	; 等到使用者鬆手，避免連續觸發
WAIT_RESET_RELEASE:	; 等待RESET放開
JNB P3.1, WAIT_RESET_RELEASE	; P3.1=0 表示還在按，持續等
SJMP GAME_LOOP	; 放開之後回到主迴圈重新跑
NO_RESET:	; 沒有按RESET的流程
; 檢查遊戲是否結束	; 如果 gameOver=1 就進入閃爍畫面
MOV A, 47H	; A ← gameOver

CJNE A, #0, GAME_OVER_BLINK ; 不等於0就跳去結束閃爍

; 顯示畫面 ; 還在遊戲中就正常顯示
ACALL DISPLAY_ALL ; 畫蛇、食物、障礙物

; 檢查按鍵 ; 讀方向鍵與速度鍵
ACALL CHECK_KEY ; 更新 nextDirX/nextDirY、調整速度

; 延遲 5ms ; 做個短延遲避免太閃
ACALL DELAY_5MS ; 呼叫 5ms 延遲

; 16-bit 計數器 t++ ; t = t + 1 (16位元溢位處理)
INC 4AH ; 先加低位
MOV A, 4AH ; A ← 低位
JNZ CHECK_SPEED ; 沒溢位就去比速度
INC 4BH ; 低位變 0 代表溢位，進位到高位

CHECK_SPEED: ; 比較 t 跟 speed
; 比較 t >= speed ; 依序比高位、低位
MOV A, 4BH ; A <- t 高位
CJNE A, 49H, CHECK_SPEED_L ; 若 A ≠ speed 高位，走另一支路
SJMP CHECK_SPEED_EQUAL_H ; 高位相等，再比低位
CHECK_SPEED_L: ; 高位不等
JC GAME_LOOP ; 若 t 高位 < speed 高位，還沒到時間
SJMP DO_MOVE ; 否則 (t 高位 > speed 高位) 一定到時間
CHECK_SPEED_EQUAL_H: ; 高位相等的情況
MOV A, 4AH ; 取 t 低位
CJNE A, 48H, CHECK_SPEED_L2 ; 比低位
SJMP DO_MOVE ; 低位也相等 -> t == speed -> 該移動

CHECK_SPEED_L2: ; 低位不等
JC GAME_LOOP ; t 低位 < speed 低位 -> 還沒到
; 若不是 JC，就表示 t 低位 > speed 低位 -> 該移動

DO_MOVE: ; 到達移動時間
; t = 0 ; 清空計時器，重新計時間
MOV 4AH, #0 ; 低位歸零
MOV 4BH, #0 ; 高位歸零

; 更新方向 ; 把下一步方向套用成當前方向
MOV A, 45H ; A <- nextDirX
MOV 43H, A ; dirX = nextDirX
MOV A, 46H ; A <- nextDirY
MOV 44H, A ; dirY = nextDirY

ACALL MOVE_SNAKE ; 真的去計算蛇的移動/碰撞/吃食物
LJMP GAME_LOOP ; 繼續下一輪主迴圈

GAME_OVER_BLINK: ; 遊戲結束閃爍模式
; 遊戲結束閃爍，按 RESET 可重新開始 ; 直到按RESET才重開
JNB P3.1, RESET_FROM_GAME_OVER ; 如果RESET按下 (P3.1=0) 就跳去重設

MOV P0, #00H ; 全亮 (或全暗) 一種狀態
MOV P1, #00H ; 行列都拉低
ACALL DELAY_LONG ; 等一下讓人看得出來
MOV P0, #0FFH ; 另一種狀態 (反相)
MOV P1, #0FFH ; 行列都拉高
ACALL DELAY_LONG ; 再延遲一下
SJMP GAME_OVER_BLINK ; 持續閃爍

RESET_FROM_GAME_OVER: ; 從 Game Over 進來的重設
ACALL INIT_GAME ; 重新初始化
WAIT_RESET_GO: ; 等使用者鬆手
JNB P3.1, WAIT_RESET_GO ; 沒放開就一直等
LJMP GAME_LOOP ; 放開就回主迴圈

; 初始化遊戲

INIT_GAME:
MOV P0, #0FFH ; 把 P0 全部設成1 (預設不點亮)
MOV P1, #0FFH ; 把 P1 全部設成1
MOV P3, #0FFH ; P3 全拉高 (按鍵未按狀態)

; Timer0 ; 設定 Timer0 當亂數來源與基準

```

MOV    TMOD, #01H          ; Timer0 模式1 (16位元)
MOV    TH0, #0             ; 初始高位清零
MOV    TL0, #0             ; 初始低位清零
SETB   TR0                 ; 啟動 Timer0

; 初始蛇身                  ; 給三節身體：X=[2,1,0]，Y 都在 3
MOV    50H, #2             ; snakeX[0]=2 (蛇頭)
MOV    70H, #3             ; snakeY[0]=3
MOV    51H, #1             ; snakeX[1]=1
MOV    71H, #3             ; snakeY[1]=3
MOV    52H, #0             ; snakeX[2]=0
MOV    72H, #3             ; snakeY[2]=3

MOV    42H, #3             ; length = 3，蛇長度3
MOV    43H, #1             ; dirX = 1，先往右走
MOV    44H, #0             ; dirY = 0，Y 不變
MOV    45H, #1             ; nextDirX = 1，下一步方向先跟現在一樣
MOV    46H, #0             ; nextDirY = 0
MOV    47H, #0             ; gameOver = 0，還沒結束
MOV    48H, #200           ; speed = 200 (低位)，基本速度 (你可以改)
MOV    49H, #0             ; speed (高位)，高位為 0
MOV    4AH, #0             ; t = 0，計數器清零
MOV    4BH, #0

ACALL  NEW_FOOD            ; 產生第一顆食物
RET                        ; 回到呼叫者

```

; 顯示整個畫面

```

DISPLAY_ALL:
; 顯示蛇身                  ; 先把蛇每一節畫出來
MOV    4CH, #0             ; i=0 (從蛇頭開始)
DISPLAY_SNAKE:
MOV    A, 4CH              ; A <- i
CJNE   A, 42H, DISPLAY_SNAKE_DO ; 若 i != length 就去畫
SJMP   DISPLAY_FOOD_PART   ; i==length 表示畫完，去顯示食物

DISPLAY_SNAKE_DO:
MOV    A, 4CH              ; A ← i
ADD    A, #50H             ; A = 50H + i -> snakeX[i] 的位址
MOV    R0, A               ; R0 指向 snakeX[i]
MOV    A, @R0              ; A ← snakeX[i]
MOV    R4, A               ; R4 = X

MOV    A, 4CH              ; 重新用 i
ADD    A, #70H             ; A = 70H + i -> snakeY[i] 的位址
MOV    R0, A               ; R0 指向 snakeY[i]
MOV    A, @R0              ; A ← snakeY[i]
MOV    R5, A               ; R5 = Y

ACALL  DISPLAY_DOT         ; 依照 (R4,R5) 這個點把 LED 亮起來

INC    4CH                 ; i++
SJMP   DISPLAY_SNAKE      ; 繼續下一節

DISPLAY_FOOD_PART:
; 顯示食物                  ; 食物也用 DISPLAY_DOT 畫
MOV    R4, 40H             ; R4 = foodX
MOV    R5, 41H             ; R5 = foodY
ACALL  DISPLAY_DOT         ; 畫食物點

; 顯示障礙物                ; 再把固定的障礙物畫上去
ACALL  DISPLAY_BLOCKS      ; 從 ROM 把 block 資料取出畫
RET                        ; 顯示完回主程式

```

; 顯示單點 (增加亮度)

```

DISPLAY_DOT:
MOV    A, #1               ; A=0000_0001，從第0列開始
MOV    R6, R4              ; R6 = X (要把 1 左轉 X 次)
DOT_X:
CJNE   R6, #0, DOT_X_DO   ; 還沒轉到位就繼續
SJMP   DOT_X_END          ; 轉好了就結束這段

```

```

DOT_X_DO:
    RL    A                      ; A 左旋轉，移到對應列的位置
    DEC   R6                    ; R6-- (還要轉幾次)
    SJMP  DOT_X
DOT_X_END:
    CPL   A                    ; 反相 (因為共陽/共陰需要反相才會亮)
    MOV   P0, A                ; 輸出到列 (P0)

    MOV   A, #1                ; 準備選行
    MOV   R6, R5               ; R6 = Y
DOT_Y:
    CJNE  R6, #0, DOT_Y_DO     ; 一樣把 bit 轉到對應行
    SJMP  DOT_Y_END
DOT_Y_DO:
    RL    A                    ; 左旋轉到指定行
    DEC   R6                   ; 次數-1
    SJMP  DOT_Y
DOT_Y_END:
    CPL   A                    ; 反相
    MOV   P1, A                ; 輸出到行 (P1)

    ; 增加顯示時間以提高亮度 (原本 R6=10，現在改為 50)      ; 小延遲讓該點亮久一點看起來比較亮
    MOV   R6, #50              ; 亮度延遲次數
DOT_DLY:
    DJNZ  R6, DOT_DLY          ; 迴圈消耗時間

    MOV   P0, #0FFH            ; 清回預設 (避免殘影)
    MOV   P1, #0FFH            ; 清回預設
    RET                        ; 回上一層

```

; 顯示障礙物 (讀取 ROM 資料)

```

DISPLAY_BLOCKS:
    MOV   DPTR, #BLOCK_DATA    ; DPTR 指向障礙資料表
    MOV   4CH, #0              ; i = 0, i=0 開始

DISP_BLOCK_LOOP:
    MOV   A, 4CH                ; A <- i
    CJNE  A, #3, DISP_BLOCK_DO ; 共有 3 組資料, i<3 就去畫
    RET                          ; 畫完三組就回去

DISP_BLOCK_DO:
    ; 取得 blockX[i]                ; 第一段是 X
    MOV   A, 4CH                ; A <- i
    MOVC  A, @A+DPTR            ; A <- CODE[DPTR+i]
    MOV   R4, A                 ; R4 = blockX[i]

    ; 取得 blockY[i]                ; 接著取得 Y (在表的後半段)
    MOV   A, 4CH                ; A <- i
    ADD   A, #3                 ; 偏移 3 (因為前面三個是 X)
    MOVC  A, @A+DPTR            ; A <- CODE[DPTR+3+i]
    MOV   R5, A                 ; R5 = blockY[i]

    ; 顯示主體                      ; 把該點畫出來
    ACALL DISPLAY_DOT           ; 畫障礙主點

    ; 顯示右側一顆 (如果 X < 7)      ; 額外在右邊補一顆方塊
    MOV   A, R4                 ; A <- X
    CJNE  A, #7, DISP_RIGHT     ; 如果 X != 7 才能往右加
    SJMP  DISP_BLOCK_NEXT       ; 已到最右邊就略過

DISP_RIGHT:
    JNC   DISP_BLOCK_NEXT       ; 若 A > 7 (其實不會) 就略過
    INC   R4                    ; X+1 (右邊那顆)
    ACALL DISPLAY_DOT           ; 再畫一次

DISP_BLOCK_NEXT:
    INC   4CH                   ; i++
    SJMP  DISP_BLOCK_LOOP       ; 下一組

```

; 檢查按鍵

```

CHECK_KEY:
    ; UP (P3.4)                ; 上鍵

```

```

JB P3.4, CHK_DOWN ; 沒按就跳過
MOV A, 44H ; 只有在目前不是往下時才能往上
CJNE A, #0, CHK_DOWN ; 若 dirY != 0 (代表上下移動中) 就不能上
MOV 45H, #0 ; nextDirX = 0
MOV 46H, #0FFH ; nextDirY = -1 (0FFH)

```

CHK_DOWN:

```

JB P3.5, CHK_LEFT ; 下鍵
MOV A, 44H ; 同理：只有不是往上時才能往下
CJNE A, #0, CHK_LEFT
MOV 45H, #0 ; nextDirX = 0
MOV 46H, #1 ; nextDirY = +1

```

CHK_LEFT:

```

JB P3.6, CHK_RIGHT ; 左鍵
MOV A, 43H ; 只有不是往右時才能往左
CJNE A, #0, CHK_RIGHT
MOV 45H, #0FFH ; nextDirX = -1
MOV 46H, #0 ; nextDirY = 0

```

CHK_RIGHT:

```

JB P3.7, CHK_SPEEDUP ; 右鍵
MOV A, 43H ; 只有不是往左時才能往右
CJNE A, #0, CHK_SPEEDUP
MOV 45H, #1 ; nextDirX = +1
MOV 46H, #0 ; nextDirY = 0

```

CHK_SPEEDUP:

```

; 加速 (P3.2) ; 處理加速鍵
JB P3.2, CHK_SPEEDDOWN ; 沒按就去看減速
; speed > 100? ; 不要變太快，設下限 100
MOV A, 49H ; 看高位
JNZ SPEED_OK_DEC ; 高位>0 一定可以減
MOV A, 48H ; 只有高位=0 才需要檢查低位
CJNE A, #100, CHK_SPEED_L1 ; 比 100
SJMP CHK_SPEEDDOWN ; 已到下限就不再減

```

CHK_SPEED_L1:

```

JC CHK_SPEEDDOWN ; 若 <100 也不減

```

SPEED_OK_DEC:

```

; speed -= 20 ; 速度數字變小 -> 實際更快
MOV A, 48H
CLR C
SUBB A, #20
MOV 48H, A
MOV A, 49H
SUBB A, #0
MOV 49H, A

```

```

; 等待放開 ; 防彈跳：等按鍵放開再回去

```

WAIT_SPEEDUP:

```

JNB P3.2, WAIT_SPEEDUP ; 還沒放就一直等

```

CHK_SPEEDDOWN:

```

; 減速 (P3.3) ; 處理減速鍵
JB P3.3, CHK_KEY_END ; 沒按就結束
; speed < 400? ; 設上限 400，避免太慢
MOV A, 49H
CJNE A, #1, CHK_SPEED_H ; 高位是否到 1
SJMP CHK_KEY_END ; 高位=1 代表已經 >=256，直接視為到上限

```

CHK_SPEED_H:

```

JC SPEED_OK_INC ; 高位 <1，代表還可以加
MOV A, 48H ; 高位>1 (>512) 其實不會到，這段做保護
CJNE A, #144, CHK_SPEED_L2 ; 檢查 400 的低位 (144)
SJMP CHK_KEY_END ; 已到 400 就不再加

```

CHK_SPEED_L2:

```

JNC CHK_KEY_END ; 若 >=144 也不加

```

SPEED_OK_INC:

```

; speed += 20 ; 速度數字變大 -> 實際更慢
MOV A, 48H

```

```
ADD A, #20
MOV 48H, A
MOV A, 49H
ADDC A, #0
MOV 49H, A
```

; 等待放開

WAIT_SPEEDDOWN:

```
JNB P3.3, WAIT_SPEEDDOWN ; 一樣防彈跳
```

CHK_KEY_END:

```
RET ; 按鍵處理結束
```

; 移動蛇

MOVE_SNAKE:

```
; newHeadX = snakeX[0] + dirX ; 新蛇頭 X
```

```
MOV A, 50H
```

```
ADD A, 43H
```

```
MOV 4DH, A ; newHeadX ; 存到暫存
```

```
; newHeadY = snakeY[0] + dirY ; 新蛇頭 Y
```

```
MOV A, 70H
```

```
ADD A, 44H
```

```
MOV 4EH, A ; newHeadY
```

```
; 檢查撞牆 ; 超出 0..7 範圍就算撞牆
```

```
MOV A, 4DH
```

```
CJNE A, #8, CHK_WALL_X ; 若 A==8 代表超右邊
```

```
SJMP COLLISION ; 撞牆
```

CHK_WALL_X:

```
JNC COLLISION ; A>=8 (不含前面等於) 也撞牆
```

```
MOV A, 4EH
```

```
CJNE A, #8, CHK_WALL_Y ; 同理檢查 Y==8
```

```
SJMP COLLISION
```

CHK_WALL_Y:

```
JNC COLLISION ; Y>=8 撞牆
```

```
; 檢查自撞，新頭跟任何一節重疊就失敗
```

```
MOV 4CH, #0
```

CHK_SELF:

```
MOV A, 4CH
```

```
CJNE A, 42H, CHK_SELF_DO ; i < length 就檢查
```

```
SJMP CHK_BLOCKS ; 全部檢查完再去查看障礙物
```

CHK_SELF_DO:

```
MOV A, 4CH
```

```
ADD A, #50H ; 取 snakeX[i]
```

```
MOV R0, A
```

```
MOV A, @R0
```

```
MOV 4FH, A
```

```
MOV A, 4DH
```

```
CJNE A, 4FH, CHK_SELF_NEXT ; X 先不同就安全
```

```
MOV A, 4CH
```

```
ADD A, #70H ; 再檢查 Y
```

```
MOV R0, A
```

```
MOV A, @R0
```

```
MOV 4FH, A
```

```
MOV A, 4EH
```

```
CJNE A, 4FH, CHK_SELF_NEXT ; Y 也不同就安全
```

```
SJMP COLLISION ; X、Y 都相同 -> 自撞
```

CHK_SELF_NEXT:

```
INC 4CH ; i++
```

```
SJMP CHK_SELF ; 繼續檢查
```

CHK_BLOCKS:

```

; 檢查撞障礙物                                ; 拿新頭去對障礙表
MOV  DPTR, #BLOCK_DATA
MOV  4CH, #0

CHK_BLOCK_LOOP:
MOV  A, 4CH
CJNE A, #3, CHK_BLOCK_DO                        ; 三顆障礙
SJMP NO_COLLISION                             ; 都沒撞到

CHK_BLOCK_DO:
MOV  A, 4CH                                ; 先取 X
MOVC A, @A+DPTR
MOV  4FH, A

MOV  A, 4DH
CJNE A, 4FH, CHK_BLOCK_NEXT                    ; X 不同就不用比 Y

MOV  A, 4CH                                ; 取對應的 Y (在表後半)
ADD  A, #3
MOVC A, @A+DPTR
MOV  4FH, A

MOV  A, 4EH
CJNE A, 4FH, CHK_BLOCK_NEXT                    ; Y 也相同 → 撞到

SJMP  COLLISION                             ; 撞障礙

CHK_BLOCK_NEXT:
INC  4CH                                ; i++
SJMP  CHK_BLOCK_LOOP                         ; 繼續檢查

COLLISION:
MOV  47H, #1                                ; gameOver = 1
RET                                           ; 回去讓主程式進入閃爍

NO_COLLISION:
; 檢查吃食物                                ; 新頭與食物同座標就代表吃到
MOV  A, 4DH
CJNE A, 40H, NO_FOOD
MOV  A, 4EH
CJNE A, 41H, NO_FOOD

; 吃到食物                                ; 長度+1，身體搬移方式不清尾巴
INC  42H                                ; length++

; 移動身體                                ; 從尾巴往前複製 (包含原尾巴)
MOV  A, 42H
DEC  A                                ; 最後一格索引 = length-1
MOV  4CH, A

COPY_GROW:
MOV  A, 4CH
CJNE A, #0, COPY_GROW_DO                      ; 還沒到頭就一直複製
SJMP  SET_NEW_HEAD                           ; 到頭了就設定新頭

COPY_GROW_DO:
MOV  A, 4CH
DEC  A                                ; 複製前一節到目前這一節 (X)
ADD  A, #50H
MOV  R0, A
MOV  A, @R0
MOV  4FH, A

MOV  A, 4CH
ADD  A, #50H
MOV  R0, A
MOV  A, 4FH
MOV  @R0, A

MOV  A, 4CH
DEC  A                                ; 複製前一節到目前這一節 (Y)

```

```
ADD A, #70H
MOV R0, A
MOV A, @R0
MOV 4FH, A
```

```
MOV A, 4CH
ADD A, #70H
MOV R0, A
MOV A, 4FH
MOV @R0, A
```

```
DEC 4CH ; i--
SJMP COPY_GROW ; 繼續往前複製
```

```
NO_FOOD:
; 移動身體 ; 沒有吃到就一般搬移（尾巴會被覆蓋掉）
MOV A, 42H
DEC A
MOV 4CH, A
```

```
COPY_NORMAL:
MOV A, 4CH
CJNE A, #0, COPY_NORMAL_DO ; 同樣從尾到頭複製
SJMP SET_NEW_HEAD
```

```
COPY_NORMAL_DO:
MOV A, 4CH
DEC A
ADD A, #50H
MOV R0, A
MOV A, @R0
MOV 4FH, A

MOV A, 4CH
ADD A, #50H
MOV R0, A
MOV A, 4FH
MOV @R0, A

MOV A, 4CH
DEC A
ADD A, #70H
MOV R0, A
MOV A, @R0
MOV 4FH, A

MOV A, 4CH
ADD A, #70H
MOV R0, A
MOV A, 4FH
MOV @R0, A

DEC 4CH
SJMP COPY_NORMAL
```

```
SET_NEW_HEAD:
; snakeX[0] = newHeadX ; 把新頭座標寫進去
MOV A, 4DH
MOV 50H, A

; snakeY[0] = newHeadY
MOV A, 4EH
MOV 70H, A

; 如果吃到食物，產生新食物 ; 再次確認（保險）
MOV A, 4DH
CJNE A, 40H, MOVE_END
MOV A, 4EH
CJNE A, 41H, MOVE_END
ACALL NEW_FOOD ; 重新隨機一顆
```

```
MOVE_END:
    RET ; 移動步驟結束
```

;**產生新食物**

NEW_FOOD:

NEW_FOOD_TRY:

```
    MOV    A, TH0 ; 亂數來源：Timer0 高位
    ADD    A, TL0 ; 加上低位
    ANL    A, #07H ; 只留低 3 bit -> 0..7
    MOV    40H, A ; foodX = A
```

```
    MOV    A, TH0 ; 再做一次（換一種混合）
    ADD    A, TL0
    RR     A ; 右移 3 次做混雜
    RR     A
    RR     A
    ANL    A, #07H
    MOV    41H, A ; foodY = A
```

;**檢查與蛇身重疊** ; 如果重疊就重抽

```
    MOV    4CH, #0
```

CHK_FOOD_SNAKE:

```
    MOV    A, 4CH
    CJNE   A, 42H, CHK_FOOD_SNAKE_DO ; i < length
    SJMP   CHK_FOOD_BLOCKS ; 檢查完蛇身，換檢查障礙
```

CHK_FOOD_SNAKE_DO:

```
    MOV    A, 4CH
    ADD    A, #50H
    MOV    R0, A
    MOV    A, @R0
    MOV    4FH, A
```

```
    MOV    A, 40H
    CJNE   A, 4FH, CHK_FOOD_SNAKE_NEXT ; X 不同就沒重疊
```

```
    MOV    A, 4CH
    ADD    A, #70H
    MOV    R0, A
    MOV    A, @R0
    MOV    4FH, A
```

```
    MOV    A, 41H
    CJNE   A, 4FH, CHK_FOOD_SNAKE_NEXT ; Y 不同就沒重疊
```

```
    SJMP   NEW_FOOD_TRY ; X、Y 都一樣 -> 重抽
```

CHK_FOOD_SNAKE_NEXT:

```
    INC    4CH
    SJMP   CHK_FOOD_SNAKE
```

CHK_FOOD_BLOCKS:

;**檢查與障礙物重疊** ; 一樣不能跟障礙重疊

```
    MOV    DPTR, #BLOCK_DATA
    MOV    4CH, #0
```

CHK_FOOD_BLOCK_LOOP:

```
    MOV    A, 4CH
    CJNE   A, #3, CHK_FOOD_BLOCK_DO ; 三組資料
    RET ; 都沒重疊 -> OK
```

CHK_FOOD_BLOCK_DO:

```
    MOV    A, 4CH
    MOVC   A, @A+DPTR
    MOV    4FH, A
```

```
    MOV    A, 40H
    CJNE   A, 4FH, CHK_FOOD_BLOCK_NEXT ; X 不同就跳過
```

```
    MOV    A, 4CH
    ADD    A, #3
```

```

MOV  A, @A+DPTR
MOV  4FH, A

MOV  A, 41H
CJNE A, 4FH, CHK_FOOD_BLOCK_NEXT      ; Y 不同也跳過

SJMP NEW_FOOD_TRY                      ; X、Y 都同 -> 重抽

CHK_FOOD_BLOCK_NEXT:
INC  4CH
SJMP  CHK_FOOD_BLOCK_LOOP

```

; 延遲子程式

```

DELAY_5MS:
MOV  R3, #50                          ; 外圈次數
D5_1: MOV  R4, #100                    ; 內圈次數
D5_2: DJNZ R4, D5_2                    ; 內圈遞減
      DJNZ R3, D5_1                    ; 外圈遞減
      RET                              ; 結束 5ms (實際時間視時脈而定)

```

```

DELAY_LONG:
MOV  R2, #60                          ; 長延遲：呼叫 5ms 共 60 次
DL:  ACALL DELAY_5MS                  ; 5ms
      DJNZ R2, DL                      ; 累積變比較久
      RET                              ; 回去

```

; 障礙物資料 (存在 CODE 區)，三個障礙的 (X陣列)(Y陣列)

```

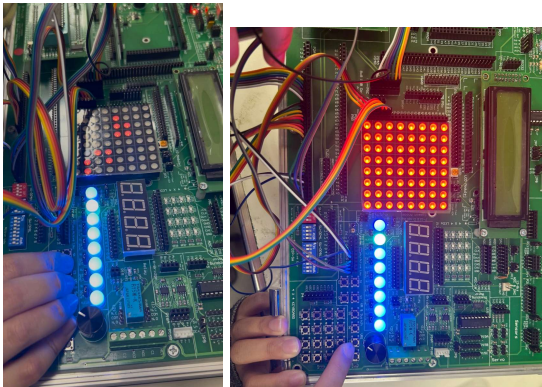
BLOCK_DATA:
DB  2, 5, 6      ; blockX[0], blockX[1], blockX[2]; X 座標 3 筆
DB  2, 4, 6      ; blockY[0], blockY[1], blockY[2]; Y 座標 3 筆

END                ; 程式結束 (組譯器識別用)

```

貪食蛇實驗照片：

左圖為正常遊玩畫面，右圖為game over畫面



兩張照片為插線(port)畫面

